

PROGRAMMATION ASSEMBLEUR x86-64

Support de cours — Licence 3 (B3)

Architecture, jeu d'instructions, pile, convention d'appel System V AMD64 et interfaçage avec C

Dr Joffrey Catena

Ce support se lit en parallèle du workbook d'exercices (.md) : chaque module de cours renvoie vers la phase d'exercices correspondante.

Sommaire

Séance 1 — 14h00–15h30 / pause / 15h45–17h15

- Module 1 — Architecture x86-64 : registres et formats de données
- Module 2 — Modes d'adressage et modèle mémoire

Séance 2 — même rythme horaire (à confirmer selon planning)

- Module 3 — Instructions arithmétiques et logiques
- Module 4 — Flux de contrôle : comparaisons, sauts, boucles

Séance 3 — même rythme horaire (à confirmer selon planning)

- Module 5 — Pile, fonctions et convention d'appel System V AMD64
- Module 6 — Appels système et interfaçage avec le C

SÉANCE 1

Créneau 1 : 14h00 – 15h30 | Pause : 15h30 – 15h45 | Créneau 2 : 15h45 – 17h15

Module 1 — Architecture x86-64 : registres et formats de données

PHASE DE COURS 1 — 14h00–14h45

1.1 Rappels : de x86 à x86-64

L'architecture x86-64 (AMD64, Intel 64) étend le jeu d'instructions x86 32 bits à des registres et des bus d'adresses 64 bits. Elle reste en grande partie rétrocompatible : le mode « long mode » 64 bits coexiste avec un mode de compatibilité 32 bits et un mode réel 16 bits historique.

- Espace d'adressage virtuel théorique : 64 bits (en pratique 48 à 57 bits selon le CPU et le mode d'extension d'adressage).
- 16 registres généraux de 64 bits, contre 8 en x86 32 bits.
- Convention d'appel standardisée (System V AMD64 ABI sous Linux/macOS, différente sous Windows).

Concrètement, cela veut dire trois choses pour vous en tant que développeur : d'abord, vous disposez de beaucoup plus de registres qu'en 32 bits, donc de moins d'allers-retours vers la mémoire ; ensuite, les adresses manipulées font 8 octets et non plus 4, ce qui change la taille des pointeurs ; enfin, il existe une règle fixe — l'ABI — qui dit dans quel registre chercher chaque argument d'une fonction, ce qui nous servira dès que nous écrirons des fonctions au Module 5.

1.2 Les registres généraux

Chaque registre 64 bits (préfixe R) possède des sous-registres 32, 16 et 8 bits accessibles indépendamment, hérités de l'historique x86.

64 bits	32 bits	16 bits	8 bits	Usage conventionnel
RAX	EAX	AX	AL	Accumulateur, valeur de retour
RBX	EBX	BX	BL	Registre de base (préservé par l'appelé)
RCX	ECX	CX	CL	Compteur (boucles, décalages)
RDX	EDX	DX	DL	Données, 2 ^e argument
RSI	ESI	SI	SIL	Source (chaînes), argument
RDI	EDI	DI	DIL	Destination (chaînes), 1 ^{er} argument
RBP	EBP	BP	BPL	Pointeur de base de pile
RSP	ESP	SP	SPL	Pointeur de sommet de pile
R8–R15	R8D–R15D	R8W–R15W	R8B–R15B	Registres supplémentaires 64 bits

Important : écrire dans un sous-registre 32 bits (ex. EAX) met automatiquement à zéro les 32 bits de poids fort du registre 64 bits correspondant (RAX). Ce n'est PAS le cas pour les écritures 16 ou 8 bits, qui préservent le reste du registre.

Petite anecdote utile en cours : les noms AX, BX, CX, DX viennent des tout premiers processeurs Intel 16 bits (années 1970-80) — Accumulator, Base, Counter, Data. Le préfixe E (Extended) est arrivé avec le 32 bits, puis le préfixe R (pour Register, 64 bits) avec l'AMD64 en 2003. R8 à R15 sont entièrement nouveaux et n'ont pas d'héritage historique, d'où leur nom purement numérique. Retenez surtout, pour l'instant, la colonne « usage conventionnel » : ce ne sont pas des règles imposées par le matériel, mais des conventions que les compilateurs et l'ABI respectent — et que

nous respecterons donc nous aussi pour que notre code assembleur reste interopérable avec du code C.

1.3 Registres spéciaux

- RIP : pointeur d'instruction (instruction pointer), non manipulable directement par MOV.
- RFLAGS : registre d'indicateurs (ZF, SF, CF, OF, PF...), positionné par les instructions arithmétiques/logiques et lu par les sauts conditionnels.
- Registres de segment (CS, DS, SS...) : largement vestigiaux en mode long, sauf FS/GS utilisés pour le TLS et certaines protections.

RIP mérite qu'on s'y arrête : c'est lui qui contient l'adresse de la prochaine instruction à exécuter, et le processeur l'incrmente tout seul après chaque instruction. On ne peut pas écrire directement dedans avec MOV, mais on peut le déplacer indirectement — c'est exactement ce que font JMP, CALL et les sauts conditionnels que nous verrons au Module 4. RFLAGS, lui, ne sert quasiment jamais à stocker une donnée utile pour vous : c'est un tableau de voyants (des bits individuels) que le processeur allume ou éteint après chaque calcul, et que l'on va lire juste après avec un saut conditionnel. Nous y reviendrons en détail au Module 3.

1.4 Tailles de données et nomenclature

Taille	Nom x86	Taille mémoire
8 bits	byte	1 octet
16 bits	word	2 octets
32 bits	dword (double word)	4 octets
64 bits	qword (quad word)	8 octets
128 bits	xmmword	16 octets (registres SSE)

Vous retrouverez ce vocabulaire (byte, word, dword, qword) directement dans le code NASM : c'est ce qu'on utilise pour dire au processeur sur combien d'octets porte une opération, par exemple avec l'instruction DB pour déclarer un byte, DW pour un word, etc. — nous les manipulons dès la section suivante.

1.5 Syntaxe : Intel vs AT&T

Deux syntaxes coexistent. Ce cours utilise la syntaxe Intel (NASM), plus lisible : destination en premier.

```
; Syntaxe Intel (NASM) – utilisée dans ce cours
mov rax, rbx          ; rax <- rbx

; Syntaxe AT&T (GAS/GCC) – pour information
movq %rbx, %rax       # rax <- rbx (source en premier, registres préfixés %)
```

1.6 Squelette d'un programme NASM sous Linux

```
section .data
    msg db "Bonjour B3", 10    ; chaîne + saut de ligne
    len equ $ - msg

section .text
    global _start

_start:
    mov rax, 1                ; syscall write
    mov rdi, 1                ; fd = stdout
```

```

mov rsi, msg      ; buffer
mov rdx, len      ; taille
syscall

mov rax, 60       ; syscall exit
mov rdi, 0
syscall

```

Assemblage et exécution :

```

nasm -f elf64 prog.asm -o prog.o
ld prog.o -o prog
./prog

```

À expliquer au tableau, ligne par ligne : le programme est découpé en deux sections. La section `.data` contient la chaîne à afficher, suivie d'un caractère 10 (le saut de ligne) ; « `len equ $ - msg` » calcule automatiquement la longueur de la chaîne pour vous, en soustrayant l'adresse de début (`msg`) de l'adresse courante (`$`). La section `.text` contient le code exécutable, et `_start` est le point d'entrée standard d'un binaire Linux — c'est par là que commence l'exécution. Le premier bloc `syscall (rax=1)` est un appel système « `write` » : on écrit `len` octets du buffer `msg` vers le descripteur de fichier 1, c'est-à-dire la sortie standard. Le second bloc (`rax=60`) est l'appel système « `exit` », qui termine proprement le programme avec le code de retour 0. Nous détaillerons la table complète des `syscalls` au Module 6 — retenez pour l'instant seulement le principe : `rax` = numéro du service demandé au noyau, les autres registres = ses arguments.

PHASE D'EXERCICES 1 — 14h45–15h30

→ **Traitez la Phase d'exercices 1 du workbook (registres, tailles, premier programme).**

PAUSE — 15h30 à 15h45

Module 2 — Modes d'adressage et modèle mémoire

PHASE DE COURS 2 — 15h45–16h30

2.1 Les sections d'un binaire

Section	Contenu
.text	Code exécutable (instructions)
.data	Données initialisées (variables globales avec valeur)
.bss	Données non initialisées (réservation d'espace, resb/resq)
.rodata	Données en lecture seule (constantes)

Ces quatre sections se retrouvent dans presque tous les binaires, quel que soit le langage d'origine (C, assembleur...) : .text est en lecture seule et exécutable, .data et .bss sont en lecture-écriture, .rodata est en lecture seule non exécutable. La différence entre .data et .bss est importante à souligner : .data occupe de la place dans le fichier binaire lui-même (puisque la valeur initiale doit être stockée quelque part), alors que .bss n'occupe aucune place dans le fichier — seule la taille à réserver est indiquée, et c'est le système qui alloue la mémoire à zéro au lancement du programme.

2.2 Modes d'adressage

L'opérande mémoire générique en x86-64 suit la forme : [base + index*échelle + déplacement].
Échelle ∈ {1, 2, 4, 8}.

Mode	Exemple	Signification
Immédiat	mov rax, 5	Charge la constante 5
Registre	mov rax, rbx	Copie rbx dans rax
Direct (mémoire)	mov rax, [var]	Charge la valeur à l'adresse var
Indirect par registre	mov rax, [rbx]	Charge la valeur à l'adresse contenue dans rbx
Base + déplacement	mov rax, [rbx+8]	Accès à un champ de structure
Base + index*échelle	mov rax, [rbx+rcx*4]	Accès à un tableau de int (4 octets)
Relatif à RIP	mov rax, [rel var]	Adressage PC-relatif (code indépendant de la position, PIE)

Le plus important à faire passer ici : les crochets [] signifient toujours « aller chercher en mémoire à cette adresse », alors que l'absence de crochets signifie « utiliser cette valeur directement ». C'est la confusion n°1 des débutants : mov rax, var charge l'adresse de var seulement si var est utilisé sans les crochets dans certains contextes (selon l'assembleur), alors que mov rax, [var] charge la valeur stockée à cette adresse. Prenez l'exemple du tableau d'entiers en base+index*échelle : si rbx pointe le début d'un tableau de int (4 octets chacun) et rcx contient l'indice i, alors [rbx+rcx*4] est exactement l'équivalent assembleur de tableau[i] en C.

2.3 LEA : calcul d'adresse sans accès mémoire

LEA (Load Effective Address) calcule une adresse selon les mêmes modes que ci-dessus mais SANS déréférencer : c'est un outil de calcul arithmétique détourné, très utilisé pour multiplier/additionner en une instruction.

```
lea rax, [rbx+rcx*2] ; rax = rbx + 2*rcx (aucun accès mémoire !)
lea rax, [rax+rax*4] ; rax = rax * 5
```

2.4 Déclaration de données

```
section .data
    a db 10 ; byte initialisé
    b dw 1000 ; word
```

```

c dd 100000          ; dword
d dq 100000000000    ; qword
tab dd 1, 2, 3, 4    ; tableau de 4 dword
msg db "texte", 0

section .bss
    buffer resb 64    ; 64 octets non initialisés
    compteur resq 1   ; 1 qword non initialisé

```

À l'oral : les lettres après le nom de la variable (db, dw, dd, dq) indiquent la taille de chaque élément — byte, word, dword, qword — et acceptent une liste de valeurs séparées par des virgules, d'où « tab dd 1, 2, 3, 4 » qui déclare un tableau de 4 entiers 32 bits contigus en mémoire. À l'inverse, resb/resw/resd/resq (dans .bss) ne prennent pas de valeur initiale mais un nombre d'éléments à réserver : « buffer resb 64 » réserve 64 octets bruts, sans les initialiser — leur contenu est indéterminé tant qu'on ne les a pas écrits explicitement.

2.5 Alignement mémoire

L'ABI System V impose que RSP soit aligné sur 16 octets au moment d'un CALL (voir Module 5). Un mauvais alignement provoque des plantages aléatoires avec les instructions SSE/AVX qui exigent un alignement 16 octets.

PHASE D'EXERCICES 2 — 16h30–17h15

→ *Traitez la Phase d'exercices 2 du workbook (adressage, LEA, structures de données).*

SÉANCE 2 — à confirmer selon planning

Créneau 1 : 14h00 – 15h30 (indicatif) | Pause : 15h30 – 15h45 | Créneau 2 : 15h45 – 17h15 (indicatif)

Module 3 — Instructions arithmétiques et logiques

PHASE DE COURS 3 — 14h00–14h45 (indicatif)

3.1 Transfert de données

Instruction	Effet
mov dst, src	dst = src
movzx dst, src	Copie avec extension par zéros (src plus petit que dst)
movsx dst, src	Copie avec extension de signe
xchg a, b	Échange a et b
push src / pop dst	Empile / dépile sur la pile

MOVZX et MOVSX servent au même problème — copier une petite valeur dans un registre plus grand — mais avec deux réponses différentes à la question « que met-on dans les bits du haut ? ». MOVZX les met à zéro : c'est ce qu'on veut pour une valeur non signée (un caractère, un booléen). MOVSX recopie le bit de signe : c'est ce qu'on veut pour préserver la valeur d'un entier signé négatif. Un exemple parlant à donner en cours : si AL contient 0xFF (soit -1 en signé sur 8 bits, ou 255 en non signé), MOVZX EAX, AL donne EAX = 0x000000FF (255), alors que MOVSX EAX, AL donne EAX = 0xFFFFFFFF (-1). Le mauvais choix entre les deux est une source d'erreurs silencieuses très classique.

3.2 Arithmétique entière

```
add rax, rbx      ; rax += rbx
sub rax, rbx      ; rax -= rbx
inc rax           ; rax += 1
dec rax           ; rax -= 1
neg rax           ; rax = -rax

imul rax, rbx     ; rax = rax * rbx (signé)
imul rax, rbx, 5   ; rax = rbx * 5

; division : RDX:RAX / opérande -> quotient dans RAX, reste dans RDX
cqo               ; étend le signe de RAX dans RDX (RDX:RAX)
idiv rbx          ; division signée par rbx
```

Attention : IDIV/DIV utilisent la paire RDX:RAX comme dividende 128 bits. Oublier d'initialiser ou d'étendre RDX (CQO/CDQ) avant une division est une erreur classique menant à une exception de division.

3.3 Logique binaire et décalages

```
and rax, rbx
or  rax, rbx
xor rax, rax      ; idiome standard pour mettre rax à 0 (plus rapide que mov rax,0)
not rax

shl rax, 1        ; décalage gauche = multiplication par 2
shr rax, 1        ; décalage droit logique (non signé)
sar rax, 1        ; décalage droit arithmétique (signé, préserve le signe)
```

La distinction SHR/SAR est un excellent point de vigilance à souligner à voix haute : les deux décalent les bits vers la droite, mais SHR insère des zéros en tête (correct pour une valeur non signée) alors que SAR recopie le bit de signe en tête (correct pour une valeur signée, car cela préserve le signe du nombre — un décalage arithmétique à droite d'un nombre négatif doit rester négatif). Utiliser SHR sur un nombre signé négatif transforme silencieusement un résultat négatif en un grand nombre positif : c'est un bug classique à mentionner explicitement aux étudiants.

3.4 Le registre RFLAGS

Indicateur	Nom	Positionné quand...
ZF	Zero Flag	Le résultat est nul
SF	Sign Flag	Le résultat est négatif (bit de poids fort = 1)
CF	Carry Flag	Retenue/emprunt en arithmétique non signée
OF	Overflow Flag	Débordement en arithmétique signée

CMP effectue une soustraction sans stocker le résultat, uniquement pour positionner les indicateurs — base des sauts conditionnels du Module 4.

PHASE D'EXERCICES 3 — 14h45–15h30 (indicatif)

→ *Traitez la Phase d'exercices 3 du workbook (arithmétique, division, manipulation de bits).*

PAUSE — 15h30 à 15h45

Module 4 — Flux de contrôle : comparaisons, sauts, boucles

PHASE DE COURS 4 — 15h45–16h30 (indicatif)

4.1 Sauts inconditionnels et conditionnels

```
jmp etiquette ; saut inconditionnel
```

Instruction	Condition (après CMP a, b)	Usage typique
je / jz	a == b (ZF=1)	égalité
jne / jnz	a != b	différence
jg / jnle	a > b (signé)	supérieur strict signé
jge / jnl	a >= b (signé)	supérieur ou égal signé
jl / jnge	a < b (signé)	inférieur strict signé
jle / jng	a <= b (signé)	inférieur ou égal signé
ja / jnbe	a > b (non signé)	supérieur strict non signé
jb / jnae	a < b (non signé)	inférieur strict non signé

Distinction essentielle : les comparaisons signées (jg/jl...) et non signées (ja/jb...) reposent sur des indicateurs différents (SF/OF vs CF). Se tromper de famille est une source classique de bugs silencieux.

4.2 Structures de contrôle classiques

Un point essentiel à marteler en cours : l'assembleur n'a ni if, ni while, ni for. Toutes ces structures que l'on connaît en C ou en Python se traduisent, au niveau machine, par la même paire d'instructions : une comparaison (CMP) suivie d'un saut conditionnel qui va sauter — ou pas — vers une étiquette. Comprendre les trois patrons ci-dessous, c'est comprendre comment n'importe quel compilateur transforme du code source en assembleur.

if / else

```

    cmp rax, rbx
    jle else_branch
    ; bloc if
    jmp end_if
else_branch:
    ; bloc else
end_if:

```

Boucle while

```

while_cond:
    cmp rcx, 10
    jge end_while
    ; corps de boucle
    inc rcx
    jmp while_cond
end_while:

```

Boucle for avec LOOP

```

    mov rcx, 10      ; compteur = nombre d'itérations
repeat:
    ; corps de boucle
    loop repeat      ; décrémente rcx, saute si rcx != 0

```

4.3 SETcc et CMOVcc

SETcc écrit 0 ou 1 dans un registre 8 bits selon une condition ; CMOVcc effectue un mouvement conditionnel sans branchement, utile pour éviter les pénalités de mauvaise prédiction de branchement.

```

cmp rax, rbx
setg cl      ; cl = 1 si rax > rbx, sinon 0

cmp rax, rbx
cmovg rdx, rax ; rdx = rax uniquement si rax > rbx

```

Pourquoi s'embêter avec CMOVcc alors qu'un simple JG ferait l'affaire ? Sur un processeur moderne, chaque saut conditionnel oblige le processeur à « deviner » à l'avance quelle branche va être prise (prédiction de branchement) pour continuer à exécuter des instructions en avance ; s'il se trompe, il doit tout annuler et recommencer, ce qui coûte une dizaine de cycles. CMOVcc évite complètement ce pari : les deux valeurs candidates sont déjà calculées, et l'instruction se contente de choisir laquelle garder, sans jamais casser le flux d'exécution. C'est un détail d'optimisation, mais qui illustre bien pourquoi comprendre ce qui se passe « sous le capot » d'un if a de la valeur, même quand on programme en langage haut niveau.

PHASE D'EXERCICES 4 — 16h30–17h15 (indicatif)

→ **Traitez la Phase d'exercices 4 du workbook (structures conditionnelles, boucles, traduction depuis du C).**

SÉANCE 3 — à confirmer selon planning

Créneau 1 : 14h00 – 15h30 (indicatif) | Pause : 15h30 – 15h45 | Créneau 2 : 15h45 – 17h15 (indicatif)

Module 5 — Pile, fonctions et convention d'appel System V AMD64

PHASE DE COURS 5 — 14h00–14h45 (indicatif)

5.1 La pile (stack)

La pile croît vers les adresses basses. RSP pointe le sommet. PUSH décrémente RSP de 8 puis écrit ; POP lit puis incrémente RSP de 8.

```
push rax    ; rsp -= 8 ; [rsp] = rax
pop  rbx    ; rbx = [rsp] ; rsp += 8
```

Un schéma simple à dessiner au tableau : représentez la mémoire avec les adresses hautes en haut et les adresses basses en bas — la pile « descend » donc visuellement vers le bas de votre schéma à chaque PUSH, et « remonte » à chaque POP. Insistez sur le fait que RSP contient une adresse, pas une valeur : PUSH RAX ne modifie pas RAX, il écrit la valeur de RAX à l'adresse RSP-8 puis met RSP à jour.

5.2 Convention d'appel System V AMD64 (Linux/macOS)

Élément	Registres / emplacement
Arguments entiers/pointeurs (dans l'ordre)	RDI, RSI, RDX, RCX, R8, R9, puis pile
Arguments flottants	XMM0–XMM7
Valeur de retour	RAX (RAX:RDX si 128 bits)
Préservés par l'appelé (callee-saved)	RBX, RBP, RSP, R12–R15
Non préservés (caller-saved)	RAX, RCX, RDX, RSI, RDI, R8–R11
Alignement RSP au moment du CALL	RSP % 16 == 0 juste avant l'instruction CALL

Deux lignes de ce tableau méritent un temps d'arrêt en cours. D'abord la distinction caller-saved / callee-saved : un registre « caller-saved » peut être librement écrasé par la fonction appelée, donc c'est à l'appelant de le sauvegarder avant l'appel s'il en a encore besoin après ; un registre « callee-saved » doit au contraire être restitué intact par la fonction appelée avant qu'elle ne retourne, donc c'est à elle de le sauvegarder puis de le restaurer si elle l'utilise. Ensuite, l'alignement de RSP sur 16 octets au moment du CALL : c'est une contrainte de l'ABI, pas du matériel — certaines fonctions C (notamment celles qui utilisent des instructions SSE/AVX en interne, comme printf) plantent ou produisent un résultat incorrect si cette règle n'est pas respectée. Nous verrons au Module 6 comment la garantir avec « and rsp, -16 ».

5.3 Prologue et épilogue de fonction

```
ma_fonction:
    push rbp                ; sauvegarde l'ancien base pointer
    mov  rbp, rsp           ; nouveau cadre de pile (stack frame)
    sub  rsp, 32            ; espace pour variables locales (multiple de 16)

    ; ... corps de la fonction, variables locales via [rbp-8], [rbp-16]...

    mov  rsp, rbp          ; libère les variables locales
    pop  rbp               ; restaure l'ancien base pointer
    ret                   ; retour (dépile l'adresse de retour posée par CALL)
```

À expliquer comme une symétrie parfaite : le prologue (push rbp / mov rbp, rsp / sub rsp, N) met en place le cadre de pile, et l'épilogue (mov rsp, rbp / pop rbp / ret) le démonte exactement dans l'ordre inverse. RBP sert de point de repère fixe pendant toute la durée de la fonction : même si RSP bouge (par des push/pop internes), RBP reste stable, ce qui permet d'accéder aux variables locales par un déplacement fixe — [rbp-8], [rbp-16], etc. — sans avoir à recalculer leur position à chaque instruction.

5.4 Exemple complet : fonction appelée depuis l'assembleur

```
section .text
global _start

addition:                ; int addition(int a, int b) -- a=edi, b=esi
    push rbp
    mov  rbp, rsp
    mov  eax, edi
    add  eax, esi        ; résultat (32 bits) dans eax
    pop  rbp
    ret

_start:
    mov  edi, 7
    mov  esi, 35
    call addition        ; addition(7, 35)
    ; eax contient 42

    mov  edi, eax        ; code de sortie = résultat
    mov  eax, 60
    syscall
```

5.5 Récursivité

Chaque appel récursif empile un nouveau cadre de pile (adresse de retour + RBP sauvegardé + locales). Sans condition d'arrêt correcte, on obtient un débordement de pile (stack overflow), détectable via la page de garde (guard page) placée par le système.

PHASE D'EXERCICES 5 — 14h45–15h30 (indicatif)

→ *Traitez la Phase d'exercices 5 du workbook (fonctions, pile, récursivité, respect de l'ABI).*

PAUSE — 15h30 à 15h45

Module 6 — Appels système et interfaçage avec le C

PHASE DE COURS 6 — 15h45–16h30 (indicatif)

6.1 Appels système Linux x86-64

Un syscall Linux se déclenche via l'instruction SYSCALL. Le numéro de service est placé dans RAX, les arguments dans RDI, RSI, RDX, R10, R8, R9 (attention : R10 remplace RCX ici, car RCX est écrasé par SYSCALL).

N° (RAX)	Syscall	Arguments
0	read	rdi=fd, rsi=buf, rdx=count
1	write	rdi=fd, rsi=buf, rdx=count

2	open	rdi=path, rsi=flags, rdx=mode
3	close	rdi=fd
9	mmap	rdi=addr, rsi=len, rdx=prot, r10=flags, r8=fd, r9=offset
60	exit	rdi=code_retour

Cette table n'est qu'un extrait des ~350 syscalls Linux x86-64, mais ces six-là couvrent la majorité des besoins d'un programme simple. La logique reste toujours la même que pour l'exemple du Module 1 : on place le numéro du service dans RAX, ses arguments dans l'ordre RDI, RSI, RDX, R10, R8, R9, puis on exécute SYSCALL. Notez bien que R10 remplace RCX pour le quatrième argument — c'est une exception à retenir, car RCX est physiquement écrasé par l'instruction SYSCALL elle-même pour stocker l'adresse de retour en mode noyau.

6.2 Appeler une fonction C depuis l'assembleur

Grâce à l'ABI commune, on peut appeler printf, malloc, etc. directement depuis NASM en respectant la convention d'appel (arguments dans RDI/RSI/RDX..., RSP aligné sur 16 avant CALL, RAX = nombre de registres XMM utilisés pour les fonctions variadiques).

```
extern printf
extern exit

section .data
    fmt db "Résultat : %d", 10, 0

section .text
global main

main:
    push rbp
    mov rbp, rsp
    and rsp, -16        ; force l'alignement 16 octets

    mov rdi, fmt        ; 1er argument : format
    mov rsi, 42         ; 2e argument : valeur
    xor eax, eax        ; 0 registre XMM utilisé (requis par printf variadique)
    call printf

    xor edi, edi
    call exit
```

Compilation avec l'éditeur de liens C (gère automatiquement le point d'entrée _start et le runtime C) :

```
nasm -f elf64 prog.asm -o prog.o
gcc prog.o -o prog
./prog
```

6.3 Appeler une fonction assembleur depuis le C

Symétriquement, une fonction assembleur respectant l'ABI (nom global, arguments en RDI/RSI..., retour en RAX) est directement appellable depuis un fichier C, en la déclarant extern côté C.

```
; carre.asm
section .text
global carre
carre:                ; int carre(int x)
    mov eax, edi
```

```

    imul eax, edi
    ret

// main.c
#include <stdio.h>
extern int carre(int x);
int main(void) {
    printf("%d\n", carre(6)); // affiche 36
    return 0;
}

nasm -f elf64 carre.asm -o carre.o
gcc main.c carre.o -o prog
./prog

```

6.4 Débogage avec GDB

- `break *adresse` ou `break etiquette` : point d'arrêt
- `stepi` / `nexti` : exécution pas à pas au niveau instruction
- `info registers` : affiche l'état des registres
- `x/8gx $rsp` : examine 8 quad-words au sommet de la pile en hexadécimal
- `disassemble` : affiche le désassemblage de la fonction courante

GDB est l'outil qui rend l'assembleur concret pour les étudiants : au lieu de deviner ce qui se passe dans leur tête, ils peuvent l'observer directement. À démontrer en direct : poser un point d'arrêt sur une fonction (`break nom_fonction`), lancer le programme (`run`), puis avancer instruction par instruction (`stepi`) en vérifiant après chaque pas l'état des registres (`info registers`) ou le contenu de la pile (`x/8gx $rsp`). C'est particulièrement efficace pour visualiser en temps réel un `push/pop` ou l'effet d'un appel de fonction sur la pile.

PHASE D'EXERCICES 6 — 16h30–17h15 (indicatif)

→ **Traitez la Phase d'exercices 6 du workbook (syscalls, interfaçage C ↔ ASM, projet de synthèse).**